

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO DỰ ÁN KHOA HỌC
Ngành: Công Nghệ Thông Tin

NGHIÊN CỨU PHƯƠNG PHÁP CHUYỂN ĐỔI OCL SANG CYPHER
VÀ ĐỒNG BỘ MÔ HÌNH UML/OCL TRÊN NEO4J SỬ DỤNG
TRIPLE GRAPH GRAMMA

Người thực hiện : Hoàng Quốc Bảo
Mã sinh viên : 23020012
Lớp : QH-I-2023/CQ-I-IT3
Giảng viên hướng dẫn : PGS TS Đặng Đức Hạnh

HANOI - 2026

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc tới thầy/cô hướng dẫn, người đã tận tình định hướng, hướng dẫn và hỗ trợ em trong suốt quá trình thực hiện khóa luận tốt nghiệp này. Những ý kiến đóng góp quý báu, kiến thức chuyên môn và phương pháp nghiên cứu khoa học mà thầy/cô chia sẻ đã giúp em hoàn thiện đề tài một cách tốt hơn.

Em xin chân thành cảm ơn các thầy cô trong Khoa Công nghệ Thông tin, Trường Đại học ..., đã truyền đạt cho em những kiến thức nền tảng và chuyên môn quý báu trong suốt quá trình học tập. Đây là cơ sở quan trọng giúp em có thể tiếp cận và thực hiện đề tài nghiên cứu này.

Em cũng xin gửi lời cảm ơn tới các anh chị, bạn bè và các thành viên trong nhóm nghiên cứu đã luôn hỗ trợ, trao đổi chuyên môn và đóng góp nhiều ý kiến hữu ích trong quá trình thực hiện đề tài.

Cuối cùng, em xin gửi lời cảm ơn sâu sắc tới gia đình, những người luôn ở bên cạnh động viên, tạo điều kiện và là nguồn động lực lớn nhất giúp em hoàn thành quá trình học tập và thực hiện khóa luận.

Mặc dù đã cố gắng hoàn thiện khóa luận trong khả năng cho phép, tuy nhiên do giới hạn về thời gian và kinh nghiệm nghiên cứu, khóa luận không tránh khỏi những thiếu sót nhất định. Em rất mong nhận được những ý kiến đóng góp từ quý thầy cô và các bạn để khóa luận được hoàn thiện hơn.

Sinh viên thực hiện

Hoàng Quốc Bảo

TÓM TẮT

Kỹ thuật phần mềm hướng mô hình (Model-Driven Engineering – MDE) nhấn mạnh vai trò trung tâm của mô hình trong quá trình phát triển hệ thống phần mềm. Trong đó, UML được sử dụng để mô tả cấu trúc và hành vi của hệ thống, trong khi OCL cung cấp cơ chế đặc tả các ràng buộc hình thức nhằm nâng cao tính chính xác ngữ nghĩa của mô hình.

Các công cụ hiện tại như USE hoặc Eclipse Modeling Framework chủ yếu xử lý mô hình trong bộ nhớ, dẫn đến những hạn chế về khả năng mở rộng đối với các mô hình có kích thước lớn. Trong khi đó, cơ sở dữ liệu đồ thị như Neo4j cung cấp khả năng lưu trữ và truy vấn hiệu quả đối với các dữ liệu có cấu trúc liên kết phức tạp.

Khóa luận này đề xuất một phương pháp chuyển đổi và đồng bộ hóa mô hình UML/OCL trên nền tảng Neo4j dựa trên Triple Graph Grammar (TGG). Nghiên cứu tập trung xây dựng cơ chế chuyển đổi ngữ nghĩa từ một tập con của OCL sang truy vấn Cypher nhằm hỗ trợ thực thi trực tiếp các ràng buộc trên cơ sở dữ liệu đồ thị. Đồng thời, nghiên cứu đề xuất cơ chế đồng bộ hai chiều và cơ chế đồng bộ tăng dần nhằm duy trì tính nhất quán giữa mô hình nguồn và mô hình đích.

Ngoài ra, nghiên cứu hiện thực một prototype tích hợp giữa USE và Neo4j nhằm kiểm chứng tính khả thi của phương pháp đề xuất. Các kết quả bước đầu cho thấy phương pháp có khả năng hỗ trợ chuyển đổi, kiểm chứng và đồng bộ hóa mô hình trên cơ sở dữ liệu đồ thị.

Do giới hạn về thời gian và phạm vi thực hiện, nghiên cứu hiện tại chưa tiến hành đánh giá trên các tập dữ liệu quy mô lớn và chưa xem xét đầy đủ các trường hợp đồng bộ phức tạp. Đây sẽ là hướng phát triển và hoàn thiện trong các nghiên cứu tiếp theo.

Từ khóa: Model-Driven Engineering, UML, OCL, Triple Graph Grammar, Neo4j, Cypher, đồng bộ tăng dần.

MỤC LỤC

LỜI CẢM ƠN	ii
TÓM TẮT	iii
MỤC LỤC	iv
DANH MỤC TỪ VIẾT TẮT	vii
DANH MỤC HÌNH VẼ	viii
DANH MỤC BẢNG BIỂU	ix
1 Giới thiệu	1
1.1 Bối cảnh nghiên cứu	1
1.1.1 MDE và vai trò của UML/OCL	1
1.1.2 Kiểm chứng OCL trên bộ nhớ	2
1.1.3 Hạn chế của USE/EMF	3
1.2 Vấn đề nghiên cứu	3
1.3 Mục tiêu nghiên cứu	4
1.4 Đóng góp của nghiên cứu	4
1.5 Cấu trúc báo cáo	5
2 Cơ sở lý thuyết và công trình liên quan	6
2.1 UML và OCL	6
2.1.1 Unified Modeling Language	6
2.1.2 Object Constraint Language	8
2.2 Triple Graph Grammar	9
2.3 Neo4j và Cypher	10
2.3.1 Neo4j và mô hình Property Graph	10
2.3.2 Ngôn ngữ truy vấn Cypher	11
2.4 Các công trình liên quan	11

2.5	Khoảng trống nghiên cứu	12
3	Phương pháp chuyển đổi và đồng bộ UML/OCL trên Neo4j	14
3.1	Kiến trúc tổng thể hệ thống	14
3.2	Biểu diễn mô hình UML/OCL trên Neo4j	16
3.2.1	Nguyên tắc ánh xạ	16
3.2.2	Phân vùng workspace	18
3.3	Chuyển đổi biểu thức OCL sang Cypher	18
3.3.1	Pipeline chuyển đổi OCL sang Cypher	18
3.3.2	Phân giải ngữ nghĩa và điều hướng	19
3.3.3	Chuyển đổi các toán tử OCL	20
3.3.4	Biểu diễn trung gian và tối ưu hóa	21
3.3.5	Sinh truy vấn Cypher	21
3.3.6	Cơ chế fallback evaluator	22
3.4	Đồng bộ mô hình dựa trên Triple Graph Grammar	23
3.4.1	Đồng bộ tiến	23
3.4.2	Đồng bộ lùi	23
3.4.3	Quản lý correspondence	24
3.5	Đồng bộ tăng dần	24
3.5.1	Phát hiện thay đổi	24
3.5.2	Lan truyền thay đổi	24
3.5.3	Phát hiện xung đột	24
4	Hiện thực và đánh giá hệ thống	26
4.1	Công nghệ và môi trường phát triển	26
4.1.1	Công nghệ sử dụng	26
4.1.2	Cấu trúc hệ thống	27
4.2	Hiện thực các thành phần hệ thống	28
4.2.1	Nạp và ánh xạ mô hình UML/TGG	28
4.2.2	Hiện thực pipeline chuyển đổi OCL	29
4.2.3	Hiện thực pipeline chuyển đổi OCL	30
4.2.4	Hệ thống chẩn đoán lỗi	31
4.3	Thiết lập thực nghiệm	31
4.3.1	Môi trường thực nghiệm	31
4.3.2	Bộ dữ liệu thực nghiệm	32
4.3.3	Kịch bản thực nghiệm	32

4.4	Kết quả thực nghiệm	33
4.4.1	Kết quả tổng quan	33
4.4.2	Kết quả semantic preservation	33
4.4.3	Kết quả chuyển đổi và đồng bộ TGG	34
4.4.4	Kết quả benchmark hiệu năng	34
4.5	Đánh giá kết quả	36
4.6	Tổng kết chương	37
4.7	Hạn chế và hướng phát triển	38
4.7.1	Hạn chế	38
4.7.2	Các môi đe dọa đến tính hợp lệ	38
4.7.3	Hướng phát triển	39
	Tài Liệu Tham Khảo	40

DANH MỤC TỪ VIẾT TẮT

Abbreviation	Meaning
BA	Business Analyst
HITL	Human-in-the-loop
IE	Information Extraction
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
QA	Question Answering
QuAM	Quantitative Argumentation Mediate
QuARS	Quality Analyzer for Requirement Specifications
RE	Relation Extraction
SemEval	Semantic Evaluation

DANH MỤC HÌNH VẼ

2.1	Mô hình UML	7
2.2	Mô hình ba thành phần của Triple Graph Grammar	9
2.3	Ví dụ mô hình Neo4j	10
3.1	Kiến trúc hệ thống	15
3.2	Kiến trúc hệ thống	16
3.3	Kiến trúc hệ thống	17
3.4	Kiến trúc hệ thống	19
3.5	Kiến trúc hệ thống	22
3.6	Kiến trúc hệ thống	22
3.7	Kiến trúc hệ thống	23

DANH MỤC BẢNG BIỂU

3.1	Ánh xạ UML sang Neo4j	17
3.2	Pipeline chuyển đổi OCL sang Cypher	19
4.1	Các công nghệ sử dụng trong hệ thống	27
4.2	Môi trường thực nghiệm	32
4.3	Tổng quan kết quả kiểm thử	33
4.4	Kết quả semantic preservation	33
4.5	Benchmark biên dịch OCL	34
4.6	Batch compilation benchmark	35
4.7	Snapshot delta benchmark	36

Chương 1

Giới thiệu

Chương này trình bày bối cảnh nghiên cứu và động lực hình thành đề tài trong lĩnh vực Kỹ thuật phần mềm hướng mô hình (Model-Driven Engineering – MDE). Nội dung chương giới thiệu vai trò của UML và OCL trong quá trình mô hình hóa và kiểm chứng hệ thống, đồng thời phân tích những hạn chế của các công cụ hiện tại như USE và Eclipse Modeling Framework (EMF) trong việc xử lý các mô hình có quy mô lớn. Từ đó, chương xác định các vấn đề nghiên cứu cần giải quyết, mục tiêu và các đóng góp chính của đề tài, đồng thời trình bày cấu trúc tổng thể của báo cáo.

1.1 Bối cảnh nghiên cứu

1.1.1 MDE và vai trò của UML/OCL

Kỹ thuật phần mềm hướng mô hình (Model-Driven Engineering – MDE) là một hướng tiếp cận trong phát triển phần mềm, trong đó mô hình được xem là hiện vật trung tâm xuyên suốt toàn bộ vòng đời hệ thống. Thay vì tập trung trực tiếp vào mã nguồn, MDE sử dụng các mô hình ở nhiều mức trừu tượng khác nhau nhằm hỗ trợ các hoạt động phân tích, thiết kế, kiểm chứng và sinh mã tự động. Việc sử dụng mô hình giúp nâng cao khả năng tái sử dụng, giảm độ phức tạp và tăng khả năng bảo trì của hệ thống phần mềm.

Trong MDE, Unified Modeling Language (UML) được sử dụng rộng rãi như một ngôn ngữ tiêu chuẩn nhằm đặc tả cấu trúc và hành vi của hệ thống. Đặc biệt, biểu đồ lớp (Class Diagram) đóng vai trò quan trọng trong việc mô hình hóa cấu trúc tĩnh thông qua các thành phần như lớp, thuộc tính, phương thức và các mối quan hệ giữa chúng. Tuy

nhiên, UML chủ yếu tập trung vào mô tả cấu trúc và chưa đủ khả năng biểu diễn đầy đủ các ràng buộc ngữ nghĩa phức tạp.

Để khắc phục hạn chế này, Object Constraint Language (OCL) được sử dụng như một ngôn ngữ đặc tả hình thức nhằm bổ sung ngữ nghĩa cho UML. OCL cho phép mô tả các quy tắc nghiệp vụ, các điều kiện tiên xử lý, hậu xử lý và các ràng buộc logic dưới dạng biểu thức khai báo [9]. Việc kết hợp UML và OCL tạo thành nền tảng quan trọng cho quá trình kiểm chứng tính nhất quán của mô hình và phát hiện sớm các lỗi thiết kế.

Ví dụ, ràng buộc OCL sau được sử dụng để đảm bảo tuổi của một sinh viên luôn lớn hơn hoặc bằng 18:

```
context Student
inv:
    self.age >= 18
```

1.1.2 Kiểm chứng OCL trên bộ nhớ

Các công cụ phổ biến như USE và EMF hiện nay chủ yếu thực hiện việc đánh giá biểu thức OCL trực tiếp trên các đối tượng được nạp vào bộ nhớ chính. Trong cách tiếp cận này, toàn bộ mô hình đối tượng được chuyển thành cấu trúc object graph trong bộ nhớ, sau đó quá trình thực thi OCL được thực hiện bằng cách điều hướng giữa các đối tượng và đánh giá các biểu thức logic tương ứng.

Ví dụ, biểu thức:

```
context Department
inv:
    employees->forall(e | e.salary > 0)
```

được xử lý bằng cách duyệt toàn bộ tập đối tượng `employees` và kiểm tra điều kiện trên từng phần tử trong tập hợp.

Cách tiếp cận này hoạt động hiệu quả đối với các mô hình nhỏ hoặc trung bình. Tuy nhiên, khi kích thước mô hình tăng lên, số lượng đối tượng và liên kết cũng tăng theo, làm cho chi phí xử lý trở nên đáng kể. Đặc biệt, các phép toán trên tập hợp như `forall()`, `exists()`, `select()` và `collect()` thường yêu cầu duyệt qua lượng lớn dữ liệu, từ đó làm gia tăng đáng kể chi phí tính toán.

1.1.3 Hạn chế của USE/EMF

Mặc dù USE và EMF đã chứng minh được hiệu quả trong quá trình mô hình hóa và kiểm chứng UML/OCL, các công cụ này vẫn tồn tại một số hạn chế đáng kể khi xử lý các mô hình lớn hoặc yêu cầu khả năng mở rộng cao.

Trước hết, các công cụ này yêu cầu toàn bộ mô hình phải được nạp vào bộ nhớ trước khi xử lý. Điều này dẫn đến việc chi phí tài nguyên tăng nhanh theo kích thước của mô hình. Khi số lượng đối tượng, liên kết hoặc ràng buộc gia tăng, hiệu năng xử lý có xu hướng suy giảm đáng kể [6].

Bên cạnh đó, các cơ chế tối ưu của các hệ quản trị cơ sở dữ liệu hiện đại như cơ chế đánh chỉ mục (indexing), lập kế hoạch truy vấn (query planning), nạp dữ liệu theo yêu cầu (lazy loading) hay tối ưu duyệt đồ thị chưa được khai thác hiệu quả trong các công cụ hiện tại. Việc đánh giá biểu thức OCL chủ yếu được thực hiện trên các đối tượng trong bộ nhớ thay vì trực tiếp trên dữ liệu được lưu trữ trong hệ quản trị dữ liệu.

Ngoài ra, các công cụ hiện tại chưa hỗ trợ đầy đủ cơ chế đồng bộ tăng dần. Khi mô hình thay đổi, phần lớn các thao tác thường phải được thực hiện lại trên toàn bộ mô hình thay vì chỉ xử lý phần dữ liệu bị ảnh hưởng trực tiếp bởi thay đổi. Điều này làm gia tăng đáng kể chi phí xử lý đối với các hệ thống có quy mô lớn và thường xuyên tiến hóa [1, 12].

Những hạn chế trên tạo động lực cho việc nghiên cứu các phương pháp kết hợp UML/OCL với cơ sở dữ liệu đồ thị nhằm tận dụng khả năng lưu trữ, truy vấn và xử lý dữ liệu quy mô lớn.

1.2 Vấn đề nghiên cứu

Từ các hạn chế của các công cụ hiện tại, nghiên cứu này tập trung giải quyết bài toán biểu diễn, chuyển đổi và đồng bộ mô hình UML/OCL trên nền tảng cơ sở dữ liệu đồ thị. Trước hết, cần xác định phương pháp biểu diễn mô hình UML/OCL trên cấu trúc đồ thị nhưng vẫn đảm bảo bảo toàn đầy đủ ngữ nghĩa của mô hình gốc.

Tiếp theo, cần xây dựng cơ chế chuyển đổi các biểu thức OCL sang ngôn ngữ truy vấn Cypher nhằm hỗ trợ thực thi và kiểm chứng ràng buộc trực tiếp trên cơ sở dữ liệu. Ngoài ra, nghiên cứu cũng xem xét bài toán đồng bộ hai chiều dựa trên Triple Graph Grammar (TGG) nhằm duy trì tính nhất quán giữa các mô hình. Đồng thời, cơ chế đồng

bộ tăng dần cần được xây dựng để giảm chi phí xử lý và tránh thực hiện lại toàn bộ quá trình đồng bộ khi xuất hiện thay đổi.

1.3 Mục tiêu nghiên cứu

Mục tiêu tổng quát của nghiên cứu là xây dựng một phương pháp chuyển đổi và đồng bộ mô hình UML/OCL trên nền tảng cơ sở dữ liệu đồ thị Neo4j nhằm nâng cao khả năng mở rộng và hiệu quả xử lý so với các công cụ truyền thống dựa trên bộ nhớ.

Để đạt được mục tiêu đó, nghiên cứu hướng tới việc xây dựng mô hình ánh xạ giữa UML/OCL và cấu trúc dữ liệu đồ thị nhằm bảo toàn ngữ nghĩa của mô hình. Đồng thời, nghiên cứu đề xuất cơ chế chuyển đổi biểu thức OCL sang truy vấn Cypher để hỗ trợ kiểm chứng trực tiếp trên cơ sở dữ liệu. Bên cạnh đó, cơ chế đồng bộ dựa trên Triple Graph Grammar được xây dựng nhằm hỗ trợ chuyển đổi hai chiều giữa các mô hình. Ngoài ra, cơ chế đồng bộ tăng dần cũng được đề xuất nhằm giảm chi phí xử lý khi mô hình thay đổi. Cuối cùng, một prototype tích hợp USE và Neo4j được phát triển nhằm kiểm chứng tính khả thi của phương pháp.

1.4 Đóng góp của nghiên cứu

Nghiên cứu này đóng góp một số kết quả chính trong việc kết hợp UML/OCL với cơ sở dữ liệu đồ thị. Trước hết, nghiên cứu đề xuất một mô hình ánh xạ hình thức giữa UML/OCL và cơ sở dữ liệu đồ thị nhằm bảo toàn ngữ nghĩa ở cả tầng mô hình và tầng đối tượng. Tiếp theo, nghiên cứu xây dựng cơ chế chuyển đổi biểu thức OCL sang truy vấn Cypher đảm bảo tính tương đương ngữ nghĩa giữa hai biểu diễn.

Ngoài ra, nghiên cứu đề xuất cơ chế đồng bộ hai chiều dựa trên Triple Graph Grammar trên nền tảng Neo4j và cơ chế đồng bộ tăng dần dựa trên phát hiện và lan truyền thay đổi nhằm giảm chi phí xử lý trên các mô hình lớn. Cuối cùng, một prototype tích hợp USE và Neo4j được xây dựng nhằm kiểm chứng tính khả thi của phương pháp đề xuất.

1.5 Cấu trúc báo cáo

Phần còn lại của báo cáo được tổ chức như sau.

Chương 2: trình bày các cơ sở lý thuyết và khảo sát các công trình nghiên cứu liên quan đến UML/OCL, Triple Graph Grammar, Neo4j, Cypher và các phương pháp đồng bộ mô hình.

Chương 3: trình bày phương pháp chuyển đổi và đồng bộ UML/OCL trên Neo4j được đề xuất trong nghiên cứu này.

Chương 4: mô tả quá trình thiết kế, hiện thực và đánh giá hệ thống nhằm kiểm chứng tính khả thi của phương pháp.

Chương 2

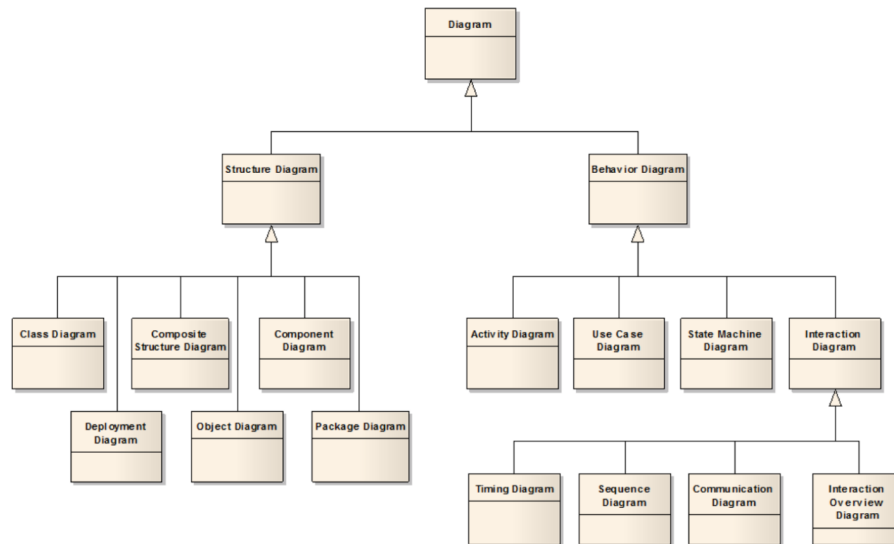
Cơ sở lý thuyết và công trình liên quan

Chương này trình bày các kiến thức nền tảng phục vụ cho quá trình xây dựng phương pháp được đề xuất trong nghiên cứu. Nội dung chương giới thiệu các khái niệm về UML, OCL, Triple Graph Grammar (TGG), cơ sở dữ liệu đồ thị Neo4j và ngôn ngữ truy vấn Cypher. Ngoài ra, chương cũng khảo sát các công trình nghiên cứu liên quan nhằm phân tích những hướng tiếp cận hiện có, từ đó xác định các khoảng trống nghiên cứu làm cơ sở cho phương pháp được đề xuất trong chương tiếp theo.

2.1 UML và OCL

2.1.1 Unified Modeling Language

Trong quá trình phát triển phần mềm, việc mô tả hệ thống bằng ngôn ngữ tự nhiên thường gây ra nhiều khó khăn do mỗi người có thể hiểu yêu cầu theo những cách khác nhau. Một yêu cầu được diễn đạt bằng văn bản đôi khi không thể hiện rõ cấu trúc dữ liệu hoặc quan hệ giữa các thành phần trong hệ thống. Vì vậy, cần có một phương pháp mô tả trực quan và thống nhất nhằm giúp những người tham gia phát triển phần mềm có thể hiểu cùng một cách.



Hình 2.1: Mô hình UML

Unified Modeling Language (UML) được phát triển bởi Object Management Group (OMG) như một ngôn ngữ mô hình hóa tiêu chuẩn nhằm hỗ trợ đặc tả, trực quan hóa và tài liệu hóa hệ thống phần mềm [10]. UML không phải là ngôn ngữ lập trình mà là ngôn ngữ dùng để xây dựng mô hình ở mức trừu tượng.

UML hỗ trợ nhiều loại biểu đồ khác nhau nhằm mô tả nhiều khía cạnh của hệ thống. Chẳng hạn, Use Case Diagram được sử dụng để mô tả yêu cầu hệ thống, Sequence Diagram dùng để mô tả tương tác giữa các thành phần, trong khi Class Diagram được dùng để mô tả cấu trúc tĩnh của hệ thống.

Trong phạm vi nghiên cứu này, biểu đồ lớp đóng vai trò quan trọng nhất do đây là thành phần trực tiếp tham gia vào quá trình chuyển đổi sang cơ sở dữ liệu đồ thị.

Biểu đồ lớp mô tả hệ thống thông qua các lớp, thuộc tính, phương thức và mối quan hệ giữa các lớp. Mỗi lớp có thể được xem như một khuôn mẫu dùng để tạo ra các đối tượng trong hệ thống.

Ví dụ, trong hệ thống quản lý thư viện có thể tồn tại lớp `Student` dùng để lưu thông tin sinh viên và lớp `Book` dùng để lưu thông tin sách. Mỗi sinh viên có thể mượn nhiều sách và mỗi cuốn sách có thể được nhiều sinh viên mượn tại các thời điểm khác nhau.

Quan hệ giữa các lớp trong UML giúp mô tả cách các thành phần liên kết với nhau. Ngoài quan hệ liên kết thông thường, UML còn hỗ trợ kế thừa, quan hệ chứa và các mối quan hệ phức tạp khác.

Tuy nhiên, UML chủ yếu tập trung vào việc mô tả cấu trúc của hệ thống. Mặc dù có thể biểu diễn được các thành phần và mối quan hệ giữa chúng, UML chưa đủ khả năng mô tả các ràng buộc nghiệp vụ hoặc điều kiện logic phức tạp.

2.1.2 Object Constraint Language

Để khắc phục hạn chế của UML, Object Constraint Language (OCL) được đưa ra như một ngôn ngữ đặc tả hình thức nhằm bổ sung thêm ngữ nghĩa cho mô hình UML [9].

Nếu UML trả lời câu hỏi hệ thống gồm những thành phần nào thì OCL trả lời câu hỏi các thành phần đó phải tuân theo điều kiện gì.

Ví dụ, trong hệ thống quản lý sinh viên có thể tồn tại quy tắc rằng mọi sinh viên đều phải có tuổi lớn hơn hoặc bằng 18. UML có thể mô tả lớp Student có thuộc tính `age`, nhưng không thể mô tả điều kiện tuổi tối thiểu. Khi đó OCL được sử dụng như sau:

```
context Student
inv:
    self.age >= 18
```

Trong biểu thức trên, từ khóa `context` xác định lớp đang được áp dụng ràng buộc, còn `inv` biểu thị điều kiện bất biến. Điều này có nghĩa rằng mọi đối tượng thuộc lớp Student đều phải thỏa mãn điều kiện trên.

Ngoài các biểu thức đơn giản, OCL còn hỗ trợ nhiều phép toán trên tập hợp.

Ví dụ:

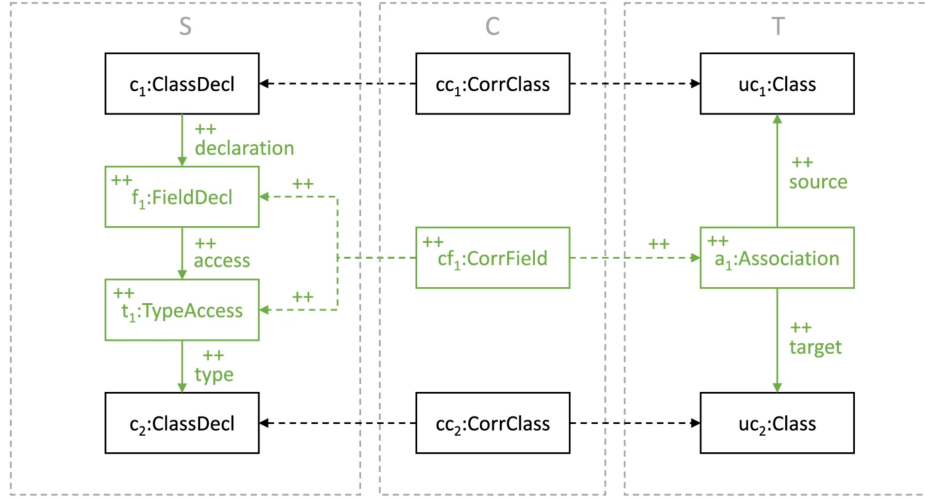
```
context Department
inv:
    employees->forall(e | e.salary > 0)
```

Biểu thức trên yêu cầu mọi nhân viên thuộc phòng ban phải có mức lương lớn hơn không.

Các phép toán như `forall()`, `exists()`, `select()` hoặc `collect()` được sử dụng phổ biến trong quá trình kiểm chứng mô hình.

Một đặc điểm quan trọng của OCL là tính chất không gây tác dụng phụ (side-effect free). Điều này có nghĩa rằng việc đánh giá biểu thức OCL chỉ dùng để kiểm tra hoặc truy vấn dữ liệu mà không làm thay đổi trạng thái của hệ thống.

2.2 Triple Graph Grammar



Hình 2.2: Mô hình ba thành phần của Triple Graph Grammar

Triple Graph Grammar (TGG) là một phương pháp được sử dụng phổ biến trong lĩnh vực chuyển đổi và đồng bộ mô hình hai chiều [5, 7].

Ý tưởng chính của TGG là sử dụng ba đồ thị để mô tả mối quan hệ giữa hai mô hình. Đồ thị thứ nhất biểu diễn mô hình nguồn, đồ thị thứ hai biểu diễn mô hình đích, trong khi đồ thị còn lại dùng để lưu trữ thông tin tương ứng giữa hai mô hình.

Có thể hình dung quá trình này giống như việc dịch ngôn ngữ. Một câu tiếng Việt có thể được chuyển thành câu tiếng Anh, và cần tồn tại một cơ chế để xác định thành phần nào của câu nguồn tương ứng với thành phần nào của câu đích.

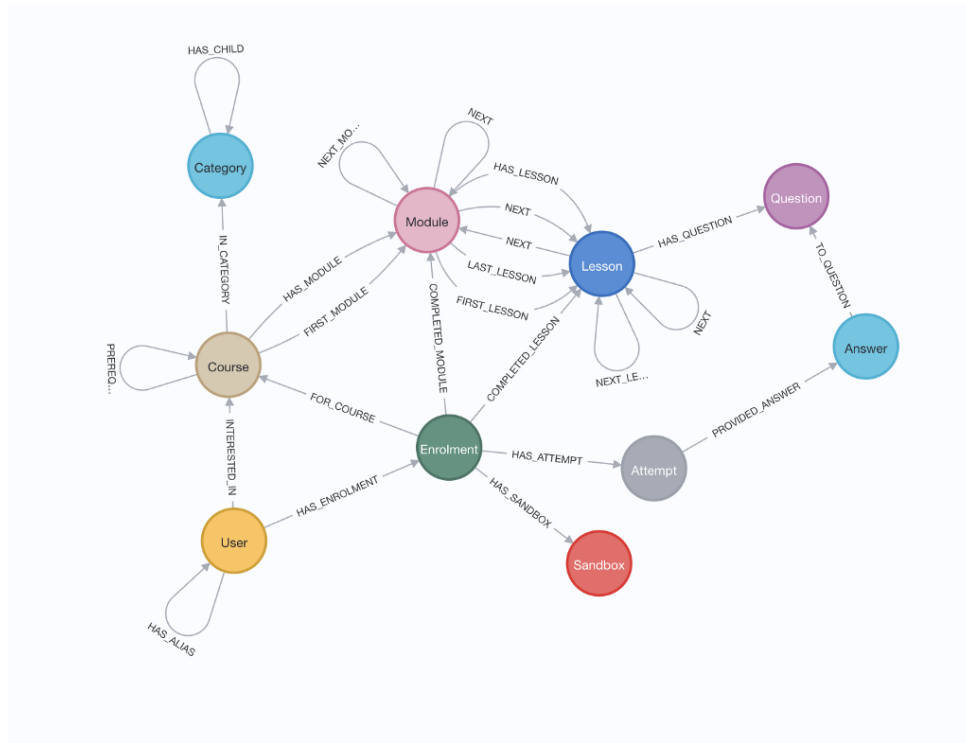
Trong TGG, phần thông tin tương ứng này được quản lý bởi Correspondence Model.

Một ưu điểm quan trọng của TGG là khả năng hỗ trợ đồng bộ hai chiều. Khi dữ liệu ở mô hình nguồn thay đổi, thay đổi đó có thể được lan truyền sang mô hình đích. Ngược lại, nếu mô hình đích thay đổi thì mô hình nguồn cũng có thể được cập nhật tương ứng.

Khả năng này đóng vai trò quan trọng trong các hệ thống cần duy trì tính nhất quán giữa nhiều biểu diễn dữ liệu khác nhau.

2.3 Neo4j và Cypher

2.3.1 Neo4j và mô hình Property Graph



Hình 2.3: Ví dụ mô hình Neo4j

Các hệ quản trị cơ sở dữ liệu quan hệ truyền thống thường lưu trữ dữ liệu dưới dạng bảng. Cách tiếp cận này hoạt động tốt đối với nhiều bài toán thông thường, tuy nhiên đối với các hệ thống có nhiều quan hệ phức tạp, việc truy vấn dữ liệu có thể trở nên khó khăn và tốn chi phí.

Cơ sở dữ liệu đồ thị được đề xuất nhằm giải quyết vấn đề đó bằng cách biểu diễn dữ liệu dưới dạng đồ thị.

Neo4j là một trong những hệ quản trị cơ sở dữ liệu đồ thị phổ biến nhất hiện nay [8].

Neo4j sử dụng mô hình Property Graph, trong đó dữ liệu được biểu diễn bằng các nút (Node) và các cạnh (Relationship).

Ví dụ, trong hệ thống quản lý thư viện, một sinh viên có thể được biểu diễn bằng một node mang nhãn Student, một cuốn sách được biểu diễn bằng node Book và quan hệ mượn sách được biểu diễn bằng cạnh Borrow.

Do UML cũng chứa nhiều thành phần và mối quan hệ tương tự nên việc ánh xạ UML sang đồ thị tương đối tự nhiên.

2.3.2 Ngôn ngữ truy vấn Cypher

Cypher là ngôn ngữ truy vấn được Neo4j sử dụng để thao tác với dữ liệu đồ thị[4]..

Khác với SQL, Cypher sử dụng cách mô tả theo mẫu đồ thị thay vì bảng dữ liệu.

Ví dụ:

```
MATCH (s:Student)
WHERE s.age >=18
RETURN s
```

Truy vấn trên trả về tất cả các sinh viên có tuổi lớn hơn hoặc bằng 18.

Khả năng mô tả trực tiếp các mẫu đồ thị giúp Cypher phù hợp với việc biểu diễn các phép điều hướng và truy vấn trong OCL.

2.4 Các công trình liên quan

Nhiều nghiên cứu trước đây đã đề xuất các phương pháp chuyển đổi UML sang cơ sở dữ liệu đồ thị nhằm tận dụng khả năng biểu diễn quan hệ của graph database [2, 3]. Tuy nhiên phần lớn các phương pháp này chủ yếu tập trung vào ánh xạ cấu trúc lớp sang node và relationship mà chưa xem xét đầy đủ ngữ nghĩa của mô hình.

Bên cạnh đó, một số nghiên cứu đã khai thác cơ sở dữ liệu đồ thị nhằm thay thế cách tiếp cận kiểm chứng mô hình truyền thống dựa trên bộ nhớ. Các hướng tiếp cận này chủ yếu tập trung vào việc lưu trữ mô hình trên graph database và sử dụng các truy vấn hoặc các luật chuyển đổi được xây dựng riêng để thực hiện kiểm chứng [2, 11]. Một số nghiên cứu cũng đã đề xuất việc thực hiện kiểm chứng mô hình trực tiếp trên Neo4j nhằm tận dụng khả năng lưu trữ và xử lý dữ liệu quy mô lớn của cơ sở dữ liệu đồ thị [11].

Tuy nhiên, các nghiên cứu hiện tại chủ yếu dừng ở mức đề xuất kiến trúc hoặc xây dựng cơ chế kiểm chứng cho các trường hợp cụ thể, thay vì tập trung vào việc chuyển đổi trực tiếp các biểu thức OCL sang truy vấn trên cơ sở dữ liệu đồ thị. Đặc biệt, hiện chưa có nhiều nghiên cứu đề xuất một phương pháp tổng quát cho việc chuyển đổi biểu

thức OCL sang truy vấn Cypher nhằm hỗ trợ thực thi và kiểm chứng trực tiếp trên Neo4j đồng thời bảo đảm tính tương đương ngữ nghĩa giữa hai biểu diễn.

Đối với đồng bộ mô hình, Triple Graph Grammar đã được áp dụng trong nhiều nghiên cứu nhằm hỗ trợ chuyển đổi hai chiều và duy trì tính nhất quán giữa các mô hình [12]. Tuy nhiên việc tích hợp trực tiếp TGG với graph database vẫn còn nhiều hạn chế.

Ngoài ra, các nghiên cứu về đồng bộ tăng dần chủ yếu tập trung vào việc phát hiện và lan truyền thay đổi nhằm giảm chi phí xử lý khi mô hình tiến hóa. Tuy nhiên, việc kết hợp cơ chế đồng bộ tăng dần với Triple Graph Grammar trên nền tảng cơ sở dữ liệu đồ thị vẫn còn hạn chế, đặc biệt trong việc xác định phạm vi dữ liệu bị ảnh hưởng và tránh xử lý lại toàn bộ mô hình.

2.5 Khoảng trống nghiên cứu

Từ việc khảo sát các công trình nghiên cứu liên quan có thể thấy rằng đã tồn tại nhiều nghiên cứu về biểu diễn mô hình UML trên cơ sở dữ liệu đồ thị cũng như các phương pháp đồng bộ mô hình dựa trên Triple Graph Grammar. Một số nghiên cứu cũng đã đề xuất việc khai thác cơ sở dữ liệu đồ thị nhằm cải thiện khả năng lưu trữ và xử lý các mô hình có kích thước lớn.

Tuy nhiên, đối với việc kiểm chứng mô hình, phần lớn các công cụ hiện tại vẫn thực hiện đánh giá các biểu thức OCL trực tiếp trên mô hình được nạp vào bộ nhớ. Mặc dù đã có các nghiên cứu đề xuất việc thực hiện kiểm chứng trên cơ sở dữ liệu đồ thị, các hướng tiếp cận này chủ yếu dừng ở mức ý tưởng hoặc xây dựng các truy vấn riêng cho từng trường hợp cụ thể. Hiện nay vẫn chưa có nhiều nghiên cứu đề xuất một phương pháp chuyển đổi tổng quát từ biểu thức OCL sang truy vấn Cypher nhằm hỗ trợ thực thi trực tiếp trên Neo4j đồng thời đảm bảo tính tương đương ngữ nghĩa.

Đối với bài toán đồng bộ mô hình, Triple Graph Grammar đã được sử dụng rộng rãi trong chuyển đổi và đồng bộ hai chiều. Tuy nhiên, khi áp dụng trên cơ sở dữ liệu đồ thị và kết hợp với cơ chế đồng bộ tăng dần, việc quản lý quá trình thực thi luật vẫn còn nhiều thách thức. Cụ thể, khi một luật đồng bộ được thực hiện nhiều lần trên các mô hình đã tồn tại hoặc trên các phần dữ liệu đã được đồng bộ trước đó, cần có cơ chế xác định các phần tử tương ứng đã được tạo trước đó nhằm tránh tạo lại đối tượng hoặc liên kết không cần thiết.

Ngoài ra, các nghiên cứu về đồng bộ tăng dần hiện chủ yếu tập trung vào việc phát

hiện và lan truyền thay đổi, trong khi việc xác định chính xác phạm vi dữ liệu bị ảnh hưởng nhằm hạn chế xử lý lại toàn bộ mô hình vẫn chưa được nghiên cứu đầy đủ trong môi trường cơ sở dữ liệu đồ thị.

Từ những khoảng trống trên có thể thấy rằng hiện chưa có một hướng tiếp cận thống nhất kết hợp việc thực thi OCL trực tiếp trên Neo4j thông qua chuyển đổi OCL sang Cypher, đồng bộ hai chiều dựa trên TGG và cơ chế đồng bộ tăng dần nhằm giảm chi phí xử lý và kiểm soát việc thực thi lặp lại của các luật đồng bộ. Đây là những vấn đề được nghiên cứu này tập trung giải quyết trong chương tiếp theo.

Chương 3

Phương pháp chuyển đổi và đồng bộ UML/OCL trên Neo4j

Chương này trình bày phương pháp biểu diễn mô hình UML/OCL trên Neo4j, pipeline chuyển đổi biểu thức OCL sang Cypher và cơ chế đồng bộ mô hình dựa trên Triple Graph Grammar (TGG). Nội dung chương tập trung vào kiến trúc tổng thể của hệ thống, các bước xử lý trong quá trình biên dịch OCL sang Cypher và cơ chế thực thi đồng bộ giữa các mô hình trên nền tảng graph database.

3.1 Kiến trúc tổng thể hệ thống

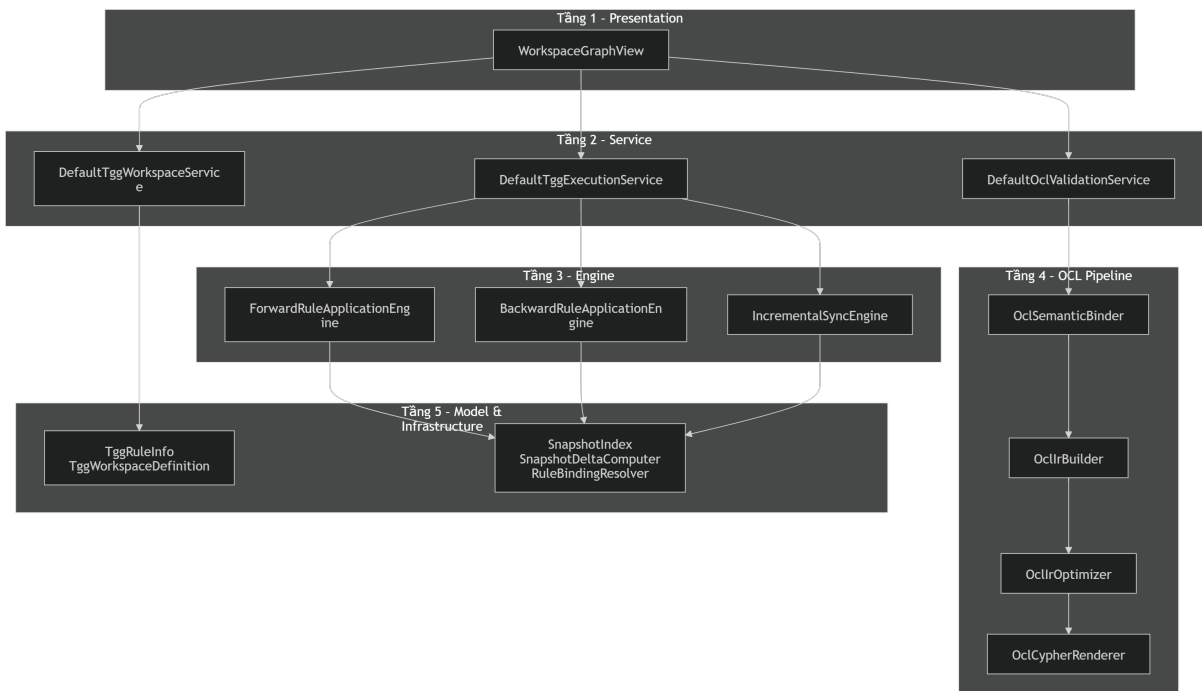
Hệ thống USE–Neo4j được xây dựng dưới dạng plugin mở rộng cho công cụ USE (UML-based Specification Environment), tích hợp Neo4j làm nền tảng lưu trữ và thực thi truy vấn đồ thị. Mục tiêu của hệ thống là hỗ trợ lưu trữ mô hình UML dưới dạng đồ thị, thực thi biểu thức OCL trực tiếp trên Neo4j và hỗ trợ đồng bộ mô hình dựa trên Triple Graph Grammar.

Kiến trúc hệ thống được tổ chức theo mô hình phân tầng nhằm tăng tính module hóa và khả năng mở rộng. Mỗi tầng đảm nhiệm một nhóm chức năng riêng biệt và giao tiếp với nhau thông qua các service trung gian.

- **Presentation Layer:** cung cấp giao diện người dùng để nạp mô hình UML/XMI/TGG, nhập biểu thức OCL, thực hiện validation và hiển thị trạng thái đồ thị Neo4j.
- **Service Layer:** điều phối các workflow chính của hệ thống như quản lý workspace,

thực thi validation OCL và điều khiển transformation engine.

- **Engine Layer:** hiện thực các cơ chế transformation và synchronization, bao gồm forward transformation, backward transformation và incremental synchronization.
- **OCL Pipeline Layer:** thực hiện pipeline chuyển đổi biểu thức OCL sang Cypher thông qua các bước parse, semantic binding, xây dựng IR và sinh truy vấn.
- **Infrastructure Layer:** hỗ trợ indexing snapshot, phát hiện thay đổi và quản lý correspondence giữa các mô hình.



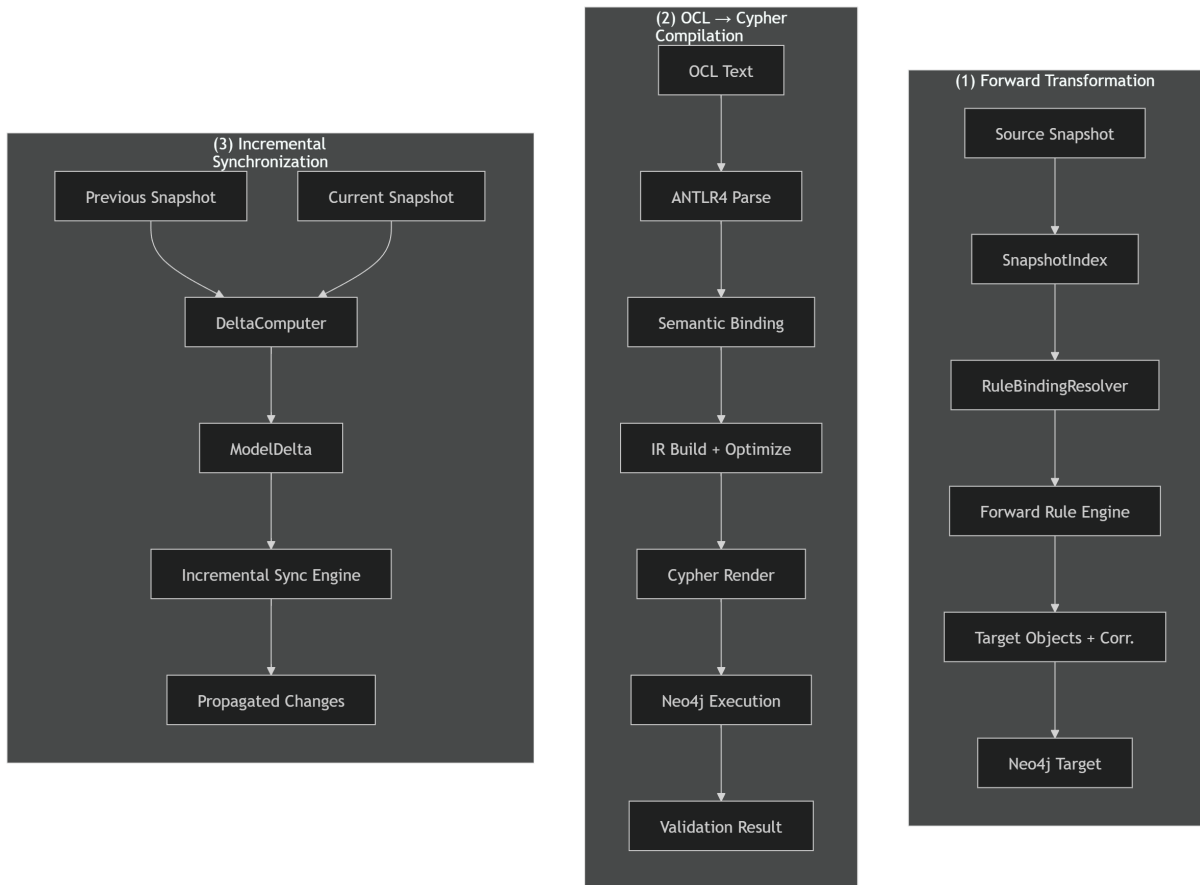
Hình 3.1: Kiến trúc hệ thống

Hệ thống hỗ trợ ba luồng xử lý chính.

Thứ nhất là **forward transformation**, trong đó mô hình nguồn được đọc từ Neo4j, các binding thỏa mãn luật TGG được tìm kiếm và mô hình đích tương ứng được sinh ra.

Thứ hai là **pipeline biên dịch OCL sang Cypher**. Biểu thức OCL được phân tích cú pháp, phân giải ngữ nghĩa và chuyển đổi thành truy vấn Cypher để thực thi trực tiếp trên Neo4j.

Thứ ba là **incremental synchronization**, cho phép hệ thống chỉ lan truyền các thay đổi cục bộ giữa hai phiên bản mô hình thay vì thực thi lại toàn bộ transformation.



Hình 3.2: Kiến trúc hệ thống

3.2 Biểu diễn mô hình UML/OCL trên Neo4j

3.2.1 Nguyên tắc ánh xạ

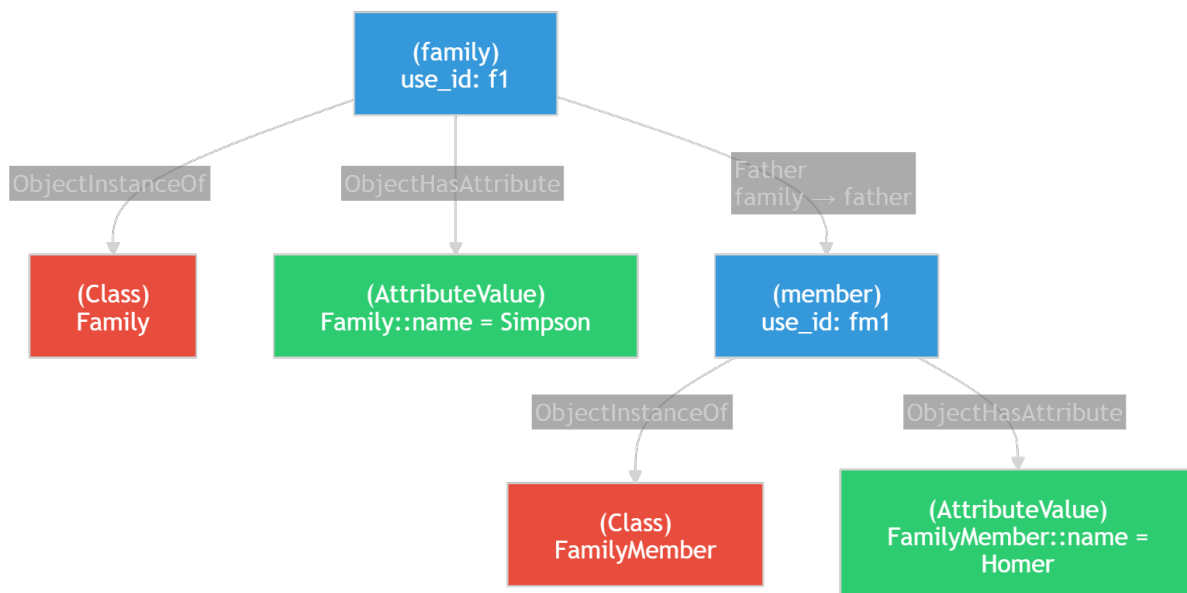
Để thực thi OCL trên Neo4j, trước tiên cần biểu diễn mô hình UML dưới dạng đồ thị. Hệ thống sử dụng một tập quy tắc ánh xạ để chuyển các phần tử UML sang các phần tử tương ứng trong Neo4j.

Bảng 3.1: Ánh xạ UML sang Neo4j

Phần tử UML	Neo4j	Mô tả
Class	Node label	Mỗi class UML tương ứng với một label trong Neo4j
Object	Node	Mỗi instance UML được biểu diễn bằng một node
Attribute	Property / Attribute node	Lưu trữ giá trị thuộc tính của object
Association	Relationship	Biểu diễn liên kết giữa các object
Inheritance	IsSubclassOf	Biểu diễn quan hệ kế thừa giữa các class

Cách biểu diễn này cho phép truy vấn Cypher khai thác trực tiếp khả năng pattern matching của Neo4j để duyệt các navigation và association trong mô hình UML.

Ví dụ, một association giữa hai đối tượng UML sẽ được biểu diễn thành một relationship trong Neo4j. Khi biểu thức OCL thực hiện navigation như `self.employee`, hệ thống có thể chuyển trực tiếp thành pattern matching trên graph database.



Hình 3.3: Kiến trúc hệ thống

3.2.2 Phân vùng workspace

Hệ thống tổ chức dữ liệu theo ba phân vùng chính tương ứng với mô hình Triple Graph Grammar.

- **Source workspace:** lưu trữ mô hình nguồn.
- **Target workspace:** lưu trữ mô hình đích.
- **Correspondence workspace:** lưu trữ correspondence objects liên kết source và target.

Correspondence workspace đóng vai trò đặc biệt quan trọng vì nó duy trì thông tin truy vết giữa các phần tử nguồn và phần tử đích. Điều này cho phép hệ thống hỗ trợ synchronization theo cả hai chiều và tránh sinh dữ liệu trùng lặp khi transformation được thực thi nhiều lần.

3.3 Chuyển đổi biểu thức OCL sang Cypher

3.3.1 Pipeline chuyển đổi OCL sang Cypher

Hệ thống đề xuất một pipeline chuyển đổi biểu thức OCL sang truy vấn Cypher để thực thi trực tiếp trên Neo4j.

Pipeline được tổ chức thành bốn giai đoạn nối tiếp nhau:

1. Phân tích cú pháp (Parsing)
2. Phân giải ngữ nghĩa (Semantic Binding)
3. Xây dựng và tối ưu biểu diễn trung gian (IR)
4. Sinh truy vấn Cypher

Bảng 3.2: Pipeline chuyển đổi OCL sang Cypher

Giai đoạn	Đầu vào	Đầu ra
Parsing	Văn bản OCL	AST
Semantic Binding	AST + Metamodel	Bound AST
IR Build + Optimize	Bound AST	Optimized IR
Cypher Rendering	IR	Cypher Query



Hình 3.4: Kiến trúc hệ thống

Ở giai đoạn đầu tiên, hệ thống sử dụng ANTLR4 để phân tích cú pháp biểu thức OCL và sinh cây cú pháp AST.

Tiếp theo, semantic binder phân giải các thành phần của biểu thức dựa trên metamodel UML, bao gồm context class, thuộc tính, navigation và iterator variable.

Sau khi semantic binding hoàn tất, biểu thức được chuyển sang biểu diễn trung gian IR nhằm tách biệt frontend OCL và backend Cypher. Điều này giúp hệ thống dễ dàng tối ưu hóa và mở rộng compiler trong tương lai.

Cuối cùng, IR được chuyển đổi thành truy vấn Cypher tương ứng để thực thi trên Neo4j.

3.3.2 Phân giải ngữ nghĩa và điều hướng

Trong giai đoạn semantic binding, hệ thống gắn thông tin kiểu cho các thành phần trong biểu thức OCL.

Các thành phần được xử lý bao gồm:

- Biến ngữ cảnh (`self`)
- Thuộc tính đối tượng

- Navigation qua association
- Iterator variables

Ví dụ, trong biểu thức:

```
self.sons->forAll(s | s.age > 18)
```

binder sẽ xác định:

- `self` thuộc kiểu `Family`
- `sons` là navigation đa trị
- `s` là iterator variable có kiểu `FamilyMember`

Việc phân giải kiểu giúp hệ thống xác định cách chuyển đổi biểu thức sang Cypher tương ứng.

3.3.3 Chuyển đổi các toán tử OCL

Hệ thống hiện hỗ trợ hơn 30 toán tử OCL phổ biến thuộc nhiều nhóm khác nhau như:

- Logic operators
- Iterator expressions
- Collection operations
- String operations
- Type checking
- Control flow expressions

Ví dụ, toán tử:

```
col->isEmpty()
```

được chuyển đổi thành:

```
size(col) = 0
```

Trong khi biểu thức:

```
col->forAll(x | P(x))
```

được chuyển đổi thành:

```
all(x IN col WHERE P(x))
```

Ngoài ra, hệ thống hỗ trợ implicit collect trong multi-valued navigation nhằm đảm bảo ngữ nghĩa của OCL khi truy cập thuộc tính qua collection.

3.3.4 Biểu diễn trung gian và tối ưu hóa

Sau semantic binding, biểu thức được chuyển sang Intermediate Representation (IR). IR đóng vai trò trung gian giữa OCL và Cypher, giúp hệ thống dễ dàng thực hiện các tối ưu hóa trước khi sinh truy vấn cuối cùng.

Hệ thống hỗ trợ một số kỹ thuật tối ưu hóa như:

- Constant folding
- Predicate simplification
- Let inlining
- Navigation optimization

Các tối ưu hóa này giúp giảm số lượng biểu thức dư thừa và tận dụng khả năng pattern matching của Neo4j để cải thiện hiệu năng thực thi truy vấn.

3.3.5 Sinh truy vấn Cypher

Sau khi tối ưu hóa IR, hệ thống sinh truy vấn Cypher tương ứng.

Mỗi invariant OCL được chuyển thành truy vấn Cypher có dạng:

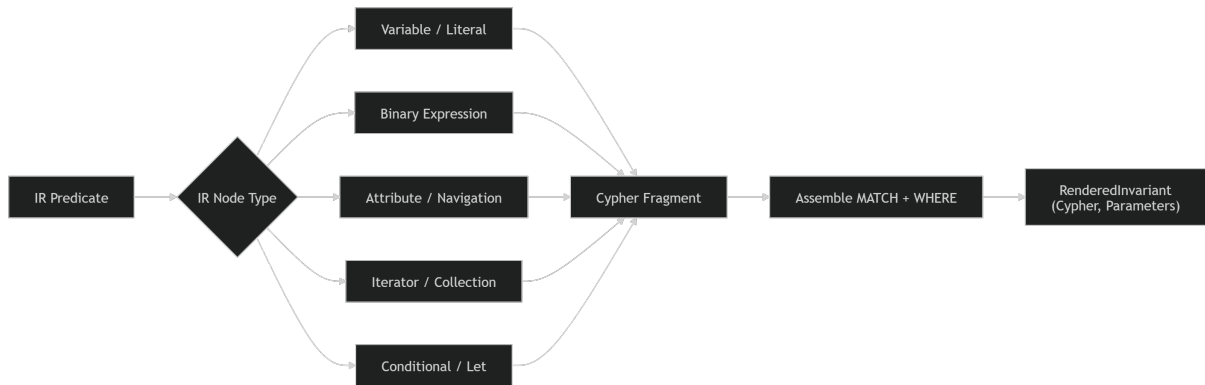
```
MATCH (self)
WHERE NOT (<predicate>)
RETURN self
```

Trong đó:

- **MATCH** tìm các đối tượng thuộc context class

- predicate là biểu thức Cypher tương đương OCL
- WHERE NOT trả về các đối tượng vi phạm invariant

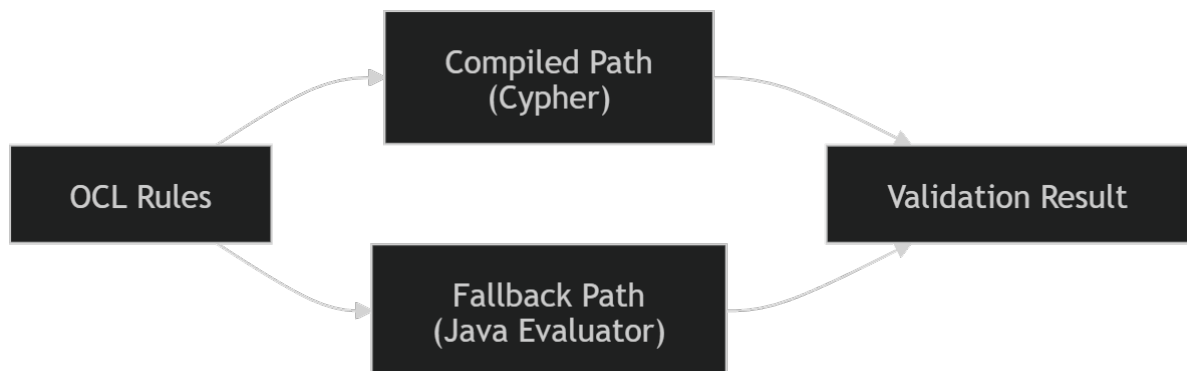
Hệ thống sử dụng parameterized query nhằm tránh injection và tận dụng query cache của Neo4j.



Hình 3.5: Kiến trúc hệ thống

3.3.6 Cơ chế fallback evaluator

Không phải mọi biểu thức OCL đều có thể được chuyển đổi hoàn toàn sang Cypher. Vì vậy, luận văn đề xuất cơ chế dual-path execution.



Hình 3.6: Kiến trúc hệ thống

Nếu compiler hỗ trợ biểu thức OCL, hệ thống sẽ sinh Cypher và thực thi trực tiếp trên Neo4j.

Ngược lại, nếu biểu thức chưa được hỗ trợ, hệ thống chuyển sang Java evaluator để đánh giá biểu thức trên từng object.

Ngoài vai trò dự phòng, fallback evaluator còn được sử dụng như oracle để đối chiếu kết quả với compiler Cypher nhằm kiểm tra semantic preservation.

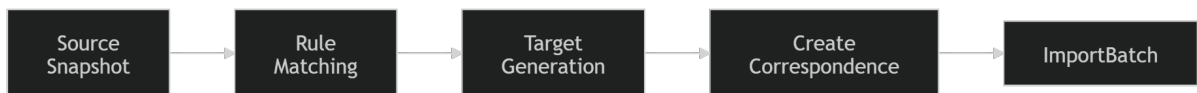
3.4 Đồng bộ mô hình dựa trên Triple Graph Grammar

3.4.1 Đồng bộ tiến

Forward transformation là quá trình sinh mô hình đích từ mô hình nguồn dựa trên các luật TGG.

Quá trình này gồm các bước:

1. Xây dựng snapshot index
2. Tìm binding cho luật TGG
3. Kiểm tra correspondence hiện có
4. Sinh object và relationship phía đích
5. Tạo correspondence mới



Hình 3.7: Kiến trúc hệ thống

Correspondence đóng vai trò duy trì liên kết giữa source object và target object, phục vụ cho synchronization và traceability.

3.4.2 Đồng bộ lùi

Backward transformation thực hiện quá trình ngược lại, tức là tái tạo mô hình nguồn từ mô hình đích.

Hệ thống sử dụng cùng cơ chế binding và correspondence như forward transformation nhưng đảo chiều xử lý giữa source và target workspace.

3.4.3 Quản lý correspondence

Để tránh sinh dữ liệu trùng lặp, hệ thống sử dụng correspondence key nhằm xác định các binding đã được áp dụng trước đó.

Cơ chế này đảm bảo tính idempotent của transformation: thực thi nhiều lần trên cùng dữ liệu sẽ không tạo thêm object trùng lặp.

3.5 Đồng bộ tăng dần

3.5.1 Phát hiện thay đổi

Đồng bộ tăng dần yêu cầu xác định chính xác các thay đổi giữa hai snapshot mô hình.

`SnapshotDeltaComputer` thực hiện:

- Phát hiện object thêm mới
- Phát hiện object bị xóa
- Phát hiện object thay đổi thuộc tính
- Phát hiện relationship thêm/xóa

Kết quả được lưu dưới dạng `ModelDelta` và sử dụng cho bước propagation tiếp theo.

3.5.2 Lan truyền thay đổi

Sau khi delta được xác định, hệ thống propagate các thay đổi sang phía đối diện thông qua correspondence.

Incremental synchronization giúp giảm đáng kể chi phí xử lý so với việc thực thi lại toàn bộ transformation, đặc biệt khi số lượng thay đổi nhỏ hơn nhiều so với kích thước toàn bộ mô hình.

3.5.3 Phát hiện xung đột

Xung đột xảy ra khi cả source và target cùng thay đổi một thuộc tính.

Hệ thống phát hiện conflict bằng cách so sánh:

- Giá trị cũ
- Giá trị mới phía source
- Giá trị hiện tại phía target

Khi phát hiện xung đột, hệ thống tạo đối tượng `SyncConflict` và ghi nhận cảnh báo trong transformation report.

Chiến lược hiện tại ưu tiên giá trị phía nguồn (`KEEP_SOURCE`), tuy nhiên cơ chế này có thể được mở rộng trong tương lai với các chiến lược conflict resolution phức tạp hơn.

Chương 4

Hiện thực và đánh giá hệ thống

Chương này trình bày quá trình hiện thực và đánh giá hệ thống USE–Neo4j được đề xuất trong nghiên cứu. Nội dung chương tập trung vào các công nghệ sử dụng, hiện thực các thành phần chính của hệ thống, môi trường thực nghiệm và các kết quả đánh giá về tính đúng đắn, khả năng bảo toàn ngữ nghĩa và hiệu năng xử lý của hệ thống.

4.1 Công nghệ và môi trường phát triển

4.1.1 Công nghệ sử dụng

Hệ thống được hiện thực chủ yếu bằng Java 17 kết hợp với Neo4j và framework USE. Java 17 được lựa chọn do hỗ trợ các đặc trưng hiện đại như sealed interface, record và pattern matching, giúp tổ chức mã nguồn rõ ràng và dễ mở rộng hơn.

Bảng 4.1: Các công nghệ sử dụng trong hệ thống

Thành phần	Công nghệ	Phiên bản	Vai trò
Ngôn ngữ	Java	17+	Hiện thực hệ thống
Build tool	Apache Maven	3.x	Quản lý dependency và build lifecycle
UML Framework	USE	7.1.0	Phân tích và quản lý mô hình UML/OCL
Parser	ANTLR4	4.x	Sinh lexer/parser cho OCL
Graph Database	Neo4j	5.x	Lưu trữ và truy vấn đồ thị
Testing	JUnit 5	5.7.0	Kiểm thử đơn vị

Hệ thống được tổ chức thành nhiều package tương ứng với các chức năng chính như pipeline OCL, transformation engine, incremental synchronization, service layer và giao diện người dùng.

4.1.2 Cấu trúc hệ thống

Module chính của hệ thống là `neo4j-tgg`, được tổ chức thành nhiều thành phần chức năng nhằm hỗ trợ quản lý mô hình UML/OCL và thực thi transformation trên Neo4j. Các thành phần chính bao gồm:

- Pipeline chuyển đổi OCL sang Cypher
- Engine thực thi TGG
- Cơ chế incremental synchronization
- Hệ thống chẩn đoán lỗi
- Giao diện quản lý workspace và validation

Pipeline OCL-to-Cypher chịu trách nhiệm phân tích, xử lý ngữ nghĩa và chuyển đổi biểu thức OCL thành truy vấn Cypher để thực thi trực tiếp trên Neo4j. Thành phần này đóng vai trò trung tâm trong cơ chế kiểm chứng ràng buộc OCL của hệ thống.

TGG engine hỗ trợ thực thi forward transformation và backward transformation giữa source model và target model. Engine này đồng thời quản lý correspondence giữa các mô hình nhằm phục vụ synchronization và traceability.

Cơ chế incremental synchronization cho phép hệ thống chỉ xử lý các thay đổi cục bộ giữa hai phiên bản mô hình thay vì thực thi lại toàn bộ transformation. Điều này giúp giảm chi phí xử lý và tăng hiệu năng trên các mô hình có kích thước lớn hơn.

Ngoài ra, hệ thống còn tích hợp cơ chế diagnostic nhằm phát hiện và phân loại các lỗi trong quá trình compile và validation OCL. Các lỗi được biểu diễn dưới dạng structured diagnostic giúp hỗ trợ debug và mở rộng compiler dễ dàng hơn.

Các thành phần trên được tổ chức độc lập theo hướng module hóa nhằm giảm sự phụ thuộc giữa các thành phần, đồng thời hỗ trợ bảo trì và mở rộng hệ thống trong tương lai.

4.2 Hiện thực các thành phần hệ thống

4.2.1 Nạp và ánh xạ mô hình UML/TGG

Hệ thống sử dụng USE Compiler để phân tích các file UML .use và tạo đối tượng metamodel chứa thông tin về class, attribute, association và inheritance trong mô hình UML. Metamodel này đóng vai trò nền tảng cho toàn bộ quá trình semantic binding, navigation resolution và validation OCL.

Từ metamodel đã nạp, hệ thống xây dựng các cấu trúc chỉ mục nhằm hỗ trợ tra cứu nhanh các class, thuộc tính và association trong quá trình xử lý biểu thức OCL. Việc xây dựng chỉ mục giúp giảm chi phí tìm kiếm và tăng hiệu năng cho pipeline biên dịch OCL sang Cypher.

Các file TGG được phân tích để tạo workspace definition, bao gồm:

- Typed variables
- Association patterns
- Predicate constraints
- Correspondence patterns

Các thành phần này được sử dụng trong quá trình tìm kiếm binding và thực thi transformation giữa source model và target model.

Ngoài ra, hệ thống hỗ trợ nạp dữ liệu XMI nhằm tạo instance model cho Neo4j workspace. Dữ liệu XMI sau khi được phân tích sẽ được chuyển đổi thành các object

snapshot và lưu trữ trên Neo4j dưới dạng node và relationship tương ứng với cấu trúc UML ban đầu.

Cách tiếp cận này giúp hệ thống duy trì được mối liên hệ giữa metamodel UML và dữ liệu đồ thị trong Neo4j, đồng thời hỗ trợ trực tiếp cho việc thực thi OCL và đồng bộ mô hình trên graph database.

4.2.2 Hiện thực pipeline chuyển đổi OCL

Pipeline chuyển đổi OCL sang Cypher được hiện thực qua bốn giai đoạn:

1. Parse biểu thức OCL
2. Semantic binding
3. Xây dựng và tối ưu IR
4. Sinh truy vấn Cypher

Trong giai đoạn semantic binding, hệ thống phân giải:

- Context class
- Navigation qua association
- Iterator variables
- Kiểu dữ liệu của biểu thức

Sau đó, biểu thức được chuyển sang Intermediate Representation (IR) nhằm tách biệt frontend OCL và backend Cypher.

Hệ thống hỗ trợ nhiều kỹ thuật tối ưu hóa như:

- Constant folding
- Condition simplification
- Let inlining
- Navigation optimization

Sau khi tối ưu hóa, truy vấn Cypher được sinh ra để thực thi trực tiếp trên Neo4j.

Ví dụ, biểu thức:

context Company

inv:

```
self.employee->forAll(e | e.age <= 65)
```

được chuyển thành truy vấn Cypher tương ứng để kiểm tra điều kiện trên graph database.

4.2.3 Hiện thực pipeline chuyển đổi OCL

Pipeline chuyển đổi OCL sang Cypher là một trong những thành phần trung tâm của hệ thống. Pipeline này được tổ chức thành bốn giai đoạn chính gồm phân tích cú pháp biểu thức OCL, semantic binding, xây dựng biểu diễn trung gian (IR) và sinh truy vấn Cypher để thực thi trên Neo4j.

Ở giai đoạn đầu tiên, biểu thức OCL được phân tích cú pháp nhằm tạo cây cú pháp AST biểu diễn cấu trúc của biểu thức. AST sau đó được sử dụng trong các bước xử lý ngữ nghĩa tiếp theo.

Tiếp theo, hệ thống thực hiện semantic binding để phân giải các thành phần trong biểu thức OCL dựa trên metamodel UML đã nạp. Quá trình này bao gồm xác định context class, phân giải navigation qua association, xử lý iterator variables và kiểm tra kiểu dữ liệu của biểu thức. Semantic binding giúp hệ thống xác định chính xác ngữ nghĩa của biểu thức trước khi chuyển đổi sang Cypher.

Sau khi semantic binding hoàn tất, biểu thức được chuyển sang Intermediate Representation (IR). IR đóng vai trò trung gian giữa frontend OCL và backend Cypher, giúp hệ thống dễ dàng thực hiện các bước tối ưu hóa và mở rộng compiler trong tương lai.

Trong giai đoạn tối ưu hóa, hệ thống áp dụng một số kỹ thuật như constant folding, condition simplification, let inlining và navigation optimization nhằm giảm độ phức tạp của biểu thức và cải thiện hiệu năng truy vấn sinh ra.

Cuối cùng, IR đã tối ưu được chuyển đổi thành truy vấn Cypher tương ứng để thực thi trực tiếp trên Neo4j.

Ví dụ, biểu thức OCL:

context Company

inv:

```
self.employee->forAll(e | e.age <= 65)
```

được chuyển thành truy vấn Cypher tương ứng nhằm kiểm tra điều kiện trên graph database.

4.2.4 Hệ thống chẩn đoán lỗi

Khi compiler gặp biểu thức OCL chưa hỗ trợ, hệ thống sử dụng structured diagnostic thay vì chỉ ném exception chung.

Các lỗi được phân loại theo nhóm:

- Lỗi ngữ nghĩa đầu vào
- Toán tử chưa hỗ trợ
- Iterator chưa hỗ trợ
- Collection operation chưa hỗ trợ
- Association không hỗ trợ

Cơ chế này giúp hệ thống phân biệt giữa lỗi mô hình và giới hạn hiện tại của compiler, từ đó quyết định sử dụng fallback evaluator hoặc báo lỗi cho người dùng.

4.3 Thiết lập thực nghiệm

4.3.1 Môi trường thực nghiệm

Các thực nghiệm được thực hiện trên môi trường Java 17 với Neo4j 5.x và USE 7.1.0.

Bảng 4.2: Môi trường thực nghiệm

OS	Windows 11
JDK	OpenJDK 17+
Build tool	Apache Maven 3.x
Testing framework	JUnit 5
UML Framework	USE 7.1.0
Graph Database	Neo4j 5.x

4.3.2 Bộ dữ liệu thực nghiệm

Nghiên cứu sử dụng nhiều bộ dữ liệu khác nhau nhằm đánh giá các thành phần của hệ thống.

Bộ dữ liệu chính là Families2Persons, bao gồm:

- Mô hình nguồn Families
- Mô hình đích Persons
- Tập luật TGG chuyển đổi giữa hai mô hình

Ngoài ra, nghiên cứu cũng sử dụng các metamodel như Company và EvalDemo để đánh giá semantic preservation và OCL validation.

Để đánh giá khả năng mở rộng, hệ thống sử dụng synthetic datasets với số lượng object và relationship tăng dần.

4.3.3 Kịch bản thực nghiệm

Các thực nghiệm tập trung vào:

- Biên dịch OCL sang Cypher
- Semantic binding và IR optimization
- Semantic preservation
- Forward/Backward transformation

- Incremental synchronization
- Snapshot delta computation
- Scalability benchmark

4.4 Kết quả thực nghiệm

4.4.1 Kết quả tổng quan

Toàn bộ test suite được thực thi thông qua Maven với kết quả:

Bảng 4.3: Tổng quan kết quả kiểm thử

Tổng số test	219
Passed	218
Failed	1
Errors	0
Tỷ lệ pass	99.54%

Failure duy nhất là một positive regression do compiler đã mở rộng hỗ trợ implicit collect vượt ngoài kỳ vọng ban đầu của test.

4.4.2 Kết quả semantic preservation

Để đánh giá bảo toàn ngữ nghĩa, hệ thống so sánh kết quả giữa Java evaluator và IR compiled evaluator trên cùng dữ liệu thực tế.

Bảng 4.4: Kết quả semantic preservation

Nhóm OCL	Java	IR	Khớp
Navigation	3	3	Yes
Iterator	3	3	Yes
Collection	6	6	Yes
Arithmetic	1	1	Yes
Logic	0	0	Yes
Control flow	0	0	Yes

Kết quả cho thấy 17/17 rules đạt mức tương đương hoàn toàn giữa Java evaluator và IR evaluator, cung cấp bằng chứng thực nghiệm rằng pipeline OCL $\rightarrow$ IR $\rightarrow$ Cypher bảo toàn ngữ nghĩa đối với các nhóm biểu thức đã hỗ trợ.

4.4.3 Kết quả chuyển đổi và đồng bộ TGG

Forward engine, backward engine và incremental synchronization đều đạt tỷ lệ pass 100% trên các test case đã xây dựng.

Các thực nghiệm cho thấy hệ thống có thể:

- Sinh target model từ source model
- Tái tạo source model từ target model
- Phát hiện và lan truyền thay đổi cục bộ
- Phát hiện conflict trong synchronization

4.4.4 Kết quả benchmark hiệu năng

4.4.4.1 OCL Compilation Benchmark

Bảng 4.5: Benchmark biên dịch OCL

Biểu thức OCL	Thời gian (ms)
employee->notEmpty()	0.40
employee->forAll(p p.age <= 65)	0.33
employee->exists(p p.age > 50)	0.23
employee->size() >= 1	0.20
Trung bình	0.23

Kết quả benchmark cho thấy thời gian biên dịch các biểu thức OCL tương đối thấp, trung bình khoảng 0.23ms cho mỗi biểu thức. Ngay cả các biểu thức có sử dụng iterator như `forAll` hay `exists` cũng được xử lý với thời gian dưới 0.5ms. Điều này cho thấy pipeline biên dịch OCL sang Cypher có chi phí xử lý nhỏ và phù hợp cho các tác vụ validation lặp lại nhiều lần. Ngoài ra, thời gian compile gần như không phụ thuộc vào kích thước dữ liệu của mô hình do quá trình biên dịch chủ yếu hoạt động trên AST và

metamodel thay vì instance graph trong Neo4j. Kết quả benchmark cũng cho thấy các bước semantic binding, xây dựng IR và sinh Cypher được thực hiện ổn định trên nhiều loại biểu thức OCL khác nhau.

4.4.4.2 Batch Compilation Benchmark

Bảng 4.6: Batch compilation benchmark

Batch	Tổng compilations	TB/compile
1×10	10	0.20ms
10×10	100	0.17ms
50×10	500	0.13ms
100×10	1000	0.09ms

Kết quả batch compilation benchmark cho thấy thời gian biên dịch trung bình trên mỗi biểu thức có xu hướng giảm khi số lượng compilation tăng lên. Với batch nhỏ, thời gian trung bình khoảng 0.20ms cho mỗi biểu thức, trong khi ở batch lớn hơn con số này giảm xuống còn khoảng 0.09ms.

Hiện tượng này chủ yếu đến từ cơ chế tối ưu của JVM, đặc biệt là hiệu ứng JIT (Just-In-Time) warm-up. Khi số lượng compilation tăng, JVM có thêm thông tin để tối ưu hóa quá trình thực thi và giảm chi phí xử lý của các đoạn mã được gọi lặp lại nhiều lần.

Kết quả benchmark cũng cho thấy pipeline biên dịch OCL hoạt động ổn định trong các kịch bản validation hàng loạt và có khả năng xử lý nhiều biểu thức liên tiếp với chi phí thấp.

4.4.4.3 Snapshot Delta Benchmark

Bảng 4.7: Snapshot delta benchmark

Objects	Links	Delta Time (ms)
100	300	3
500	1500	2
1000	3000	3
5000	15000	12
10000	30000	25

Kết quả benchmark cho thấy thời gian tính toán delta có xu hướng tăng theo kích thước mô hình, tuy nhiên mức tăng vẫn tương đối ổn định và chưa xuất hiện hiện tượng tăng trưởng bùng nổ. Với các bộ dữ liệu nhỏ và trung bình, thời gian xử lý chỉ dao động trong khoảng từ 2ms đến 3ms.

Ngay cả với bộ dữ liệu lớn nhất gồm 10,000 objects và 30,000 links, hệ thống vẫn hoàn thành quá trình phát hiện thay đổi trong khoảng 25ms. Điều này cho thấy cơ chế snapshot indexing và delta computation của hệ thống có chi phí xử lý tương đối thấp đối với các kịch bản synchronization thông thường.

Kết quả benchmark cũng cho thấy incremental synchronization có tiềm năng mở rộng tốt hơn so với việc thực thi lại toàn bộ transformation, đặc biệt trong các trường hợp số lượng thay đổi nhỏ hơn nhiều so với kích thước tổng thể của mô hình.

4.5 Đánh giá kết quả

Kết quả thực nghiệm cho thấy hệ thống đạt mức ổn định cao với tỷ lệ pass 99.54%.

Pipeline OCL-to-Cypher hỗ trợ hơn 30 toán tử OCL và đạt 100% semantic consistency giữa Java evaluator và IR evaluator trên các nhóm biểu thức được kiểm thử.

Forward transformation, backward transformation và incremental synchronization đều hoạt động đúng trên các test case đã xây dựng.

Các benchmark cũng cho thấy hệ thống có hiệu năng tốt đối với các tác vụ:

- Biên dịch OCL

- Snapshot delta computation
- Incremental synchronization

Tuy nhiên, hệ thống hiện vẫn còn một số hạn chế:

- Chưa hỗ trợ n-ary association và qualified association
- Conflict resolution mới dừng ở mức phát hiện conflict
- Chưa thực hiện benchmark trên tập dữ liệu rất lớn hoặc Neo4j cluster

4.6 Tổng kết chương

Chương này đã trình bày quá trình hiện thực và đánh giá hệ thống USE–Neo4j.

Về hiện thực, hệ thống đã xây dựng thành công pipeline chuyển đổi OCL sang Cypher, engine chuyển đổi TGG hai chiều và cơ chế incremental synchronization trên Neo4j.

Về thực nghiệm, hệ thống đạt tỷ lệ pass 99.54% trên 219 test cases và đạt 100% semantic consistency giữa Java evaluator và IR evaluator.

Các benchmark cho thấy hệ thống có hiệu năng tốt đối với biên dịch OCL và xử lý delta trên các bộ dữ liệu có quy mô tăng dần.

4.7 Hạn chế và hướng phát triển

Mặc dù đạt được các mục tiêu đặt ra, nghiên cứu hiện tại vẫn tồn tại một số hạn chế nhất định.

4.7.1 Hạn chế

Mặc dù hệ thống đã hỗ trợ nhiều nhóm biểu thức OCL phổ biến và thực hiện thành công việc chuyển đổi sang Cypher, nghiên cứu hiện tại vẫn còn một số hạn chế nhất định. Trước hết, phạm vi hỗ trợ OCL hiện mới tập trung vào các biểu thức thường dùng như navigation, iterator, collection operation và control flow cơ bản. Một số đặc trưng nâng cao của OCL như `closure`, `recursive expression` hoặc `pre/post-condition` hiện chưa được hỗ trợ đầy đủ.

Bên cạnh đó, các thực nghiệm hiện tại chủ yếu được thực hiện trên các case study có quy mô nhỏ và trung bình. Mặc dù kết quả bước đầu cho thấy hệ thống hoạt động ổn định và có khả năng mở rộng tương đối tốt, nghiên cứu vẫn chưa đánh giá trên các tập dữ liệu rất lớn hoặc môi trường phân tán thực tế.

Ngoài ra, nghiên cứu hiện chưa thực hiện so sánh định lượng với các công cụ hoặc phương pháp khác liên quan đến thực thi OCL hoặc synchronization trên graph database. Điều này khiến việc đánh giá mức độ cải thiện về hiệu năng hoặc khả năng mở rộng vẫn còn hạn chế.

Đối với incremental synchronization, hệ thống hiện mới tập trung vào cơ chế phát hiện và lan truyền thay đổi giữa source model và target model. Các vấn đề như đồng bộ song song, xử lý xung đột phức tạp hoặc synchronization trong môi trường phân tán hiện chưa được xem xét đầy đủ.

4.7.2 Các mối đe dọa đến tính hợp lệ

Một số yếu tố có thể ảnh hưởng đến tính đúng đắn và khả năng khái quát hóa của kết quả nghiên cứu.

Thứ nhất, các bộ dữ liệu thực nghiệm hiện tại chủ yếu được xây dựng từ các case study và synthetic datasets, do đó có thể chưa phản ánh đầy đủ các đặc điểm của các hệ thống thực tế có quy mô lớn và cấu trúc phức tạp hơn.

Thứ hai, số lượng case study được sử dụng trong thực nghiệm còn tương đối hạn chế. Điều này có thể ảnh hưởng đến khả năng đánh giá đầy đủ phạm vi hỗ trợ OCL cũng như tính ổn định của transformation engine trên nhiều loại mô hình khác nhau.

Ngoài ra, các benchmark hiện được thực hiện trên môi trường máy đơn với quy mô dữ liệu tăng dần. Hiệu năng thực tế của hệ thống có thể thay đổi khi kích thước mô hình tiếp tục tăng hoặc khi triển khai trong môi trường Neo4j cluster và distributed graph database.

4.7.3 Hướng phát triển

Trong tương lai, nghiên cứu có thể được mở rộng theo nhiều hướng nhằm cải thiện khả năng hỗ trợ OCL, hiệu năng xử lý và khả năng mở rộng của hệ thống.

Một hướng phát triển quan trọng là mở rộng phạm vi hỗ trợ của compiler để bao phủ đầy đủ hơn các thành phần của OCL, đặc biệt là các biểu thức nâng cao như recursive expression, closure và pre/post-condition.

Bên cạnh đó, quá trình sinh truy vấn Cypher có thể được tối ưu thêm nhằm giảm số lượng pattern matching dư thừa và tận dụng tốt hơn cơ chế query planning của Neo4j.

Về thực nghiệm, hệ thống cần được đánh giá trên các tập dữ liệu lớn hơn và các case study thực tế nhằm kiểm tra khả năng mở rộng trong môi trường công nghiệp.

Ngoài ra, nghiên cứu cũng có thể mở rộng theo hướng hỗ trợ synchronization song song, xử lý xung đột nâng cao và incremental synchronization trong môi trường phân tán.

Cuối cùng, hệ thống có thể được mở rộng để hỗ trợ thêm các graph database khác ngoài Neo4j nhằm tăng tính linh hoạt và khả năng ứng dụng của giải pháp.

Tài Liệu Tham Khảo

- [1] R. Bill, M. Gogolla, and M. Wimmer, “On leveraging UML/OCL for model synchronization,” pp. 20–29, accepted: 2022-08-04T16:29:26Z. [Online]. Available: <https://repositum.tuwien.at/handle/20.500.12708/56553>
- [2] G. Daniel, F. Jouault, G. Sunyé, and J. Cabot, “Gremlin-ATL: A scalable model transformation framework.” [Online]. Available: <https://hal.science/hal-01589582>
- [3] G. Daniel, G. Sunyé, and J. Cabot, “Umltographdb: Mapping conceptual schemas to graph databases,” in *Conceptual Modeling*, ser. Lecture Notes in Computer Science, vol. 9974. Springer, 2016, pp. 430–444.
- [4] N. Francis *et al.*, “Formal semantics of the language cypher,” *arXiv preprint arXiv:1802.09984*, 2018.
- [5] F. Hermann, H. Ehrig, F. Orejas, K. Czarnecki, Z. Diskin, Y. Xiong, S. Gottmann, and T. Engel, “Model synchronization based on triple graph grammars: correctness, completeness and invertibility,” vol. 14, no. 1, pp. 241–269. [Online]. Available: <http://link.springer.com/10.1007/s10270-012-0309-1>
- [6] M. Koegel and J. Helming, “What every eclipse developer should know about emf,” <https://eclipse.dev/emf/docs.html>, 2014.
- [7] E. Leblebici, A. Anjorin, A. Schürr, and G. Taentzer, “Multi-amalgamated triple graph grammars,” pp. 87–103.
- [8] Neo4j Inc., “Neo4j documentation,” 2026. [Online]. Available: <https://neo4j.com/docs/>
- [9] Object Management Group, “Object constraint language,” OMG, Tech. Rep. formal/10-02-01, 2010. [Online]. Available: <https://www.omg.org/spec/OCL/2.2/PDF>

- [10] —, “Omg unified modeling language version 2.5.1,” <https://www.omg.org/spec/UML/2.5.1/PDF>, 2017, version 2.5.1.
- [11] C. Praschl, A. M. Pointner, O. Krauss, E. Helm, and A. H. Schuler, “Model verification in graph databases and its application in neo4j: European modeling & simulation symposium 2022.” [Online]. Available: <https://www.scopus.com/pages/publications/85142913797>
- [12] N. Weidmann and A. Anjorin, “eMoflon::neo - consistency and model management with graph databases.”